READING SHEET

Frontend Capstone — Build

Module 3 · Frontend: React & Next.js · Day 49 · Week 10

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	**Capstone build**	**The full session**

> **Tip How to use this sheet**
> A working document for the last build day. Part 1 is what is being assessed. Part 2 is the triage method — do it in the first half hour and write the plan down. Part 3 is the full verification battery, run against the deployed URL. Part 4 is tomorrow's presentation, which you should rehearse aloud today. Do not change the application after tonight.

Part 1 · The Standard

The capstone is judged on whether the whole span of two modules is present and working together in one application. **Breadth, not novelty** — nothing new is expected today, and a small, complete, well-built project beats an ambitious half-finished one every time.

FROM	WHAT IT MUST SHOW
Module 2	Semantic markup, responsive layout, modern JavaScript
Weeks 6–7	Components, state, hooks, routing, forms
Week 8	Layouts, rendering strategy, the server boundary
Week 9	Authentication, data fetching, performance, metadata
Week 10	Data presentation, formatting, deployment

How it is marked

CRITERION	WEIGHT
Functionality and completeness	25%
Architecture and code quality	20%
Authentication and data handling	20%
Performance, accessibility, metadata	15%
Deployment and documentation	10%
Presentation and code walkthrough	10%

> **Warning A fifth of the mark is not code**
> Deployment, documentation and the walkthrough together carry twenty per cent. They are also the cheapest marks in the rubric and the ones most often left until it is too late. The pass mark is 70%, as everywhere in the program.

What "done" looks like

NOT DONE	DONE
Runs on your laptop	A live URL anyone can open
A route that renders	A route that handles no data
Signed-in pages look right	The three attacks all fail
"It should be fast"	Numbers in PERF.md
A README with the project name	A README someone can follow
You can demonstrate it	Someone else can use it

The last line is the real test. Hand your phone to the person beside you without instructions and watch — whatever they hesitate over is the most valuable bug report you will get today. And empty states count as features: a screen that breaks when there is no data is not finished, and an assessor *will* create a new account tomorrow.

The four documents

FILE	WHAT IT CONTAINS
`README.md`	What it is, the live URL, setup from a fresh clone, variables
`AUTH.md`	Every private route, its layers, what each proves, the three attacks
`DATA.md`	Every query: server or client, key, refresh rule, reasoning
`PERF.md`	The budget with both columns, from the deployed build

Write them **today**, while the decisions are fresh. Each takes fifteen minutes now and an hour tomorrow, and together they are ten per cent of the mark for less than an hour of work. A missing README is the most common avoidable loss on this rubric.

Part 2 · Triage & Plan

Four hours, and almost everyone has more outstanding than fits in them. The skill being taught today is not building faster — it is choosing correctly what *not* to build.

LIST	MEANING	EXAMPLE
Must	Incomplete without it	Sign-in works; the main flow completes
Should	Visibly missing, still works	A dashboard chart; the empty states
Could	Nobody will notice its absence	An animation; a second theme

> **Tip Then be honest, and cut out loud**
> Most students put too much in Must. The test: if you removed it, could someone still complete the main task the application exists for? If yes, it is not a Must. Delete the Coulds from the list into a FUTURE.md — an idea kept "just in case" quietly consumes attention all day, and naming what you left out reads as judgement where silence reads as oversight.

The order to finish in

#	FINISH IN THIS ORDER
1	Anything broken — fix before you add
2	The main flow, end to end, on the live URL
3	Authentication and the three attacks
4	Empty, loading and error states everywhere
5	The Shoulds, largest value first
6	Documents, README, and a rehearsal

Never start a feature after the halfway point. A half-built feature scores worse than an absent one, because it makes the application look broken. Past two hours, only finish, verify and document. Push after every meaningful piece rather than once at the end — a broken deployment at 15:45 with one commit to inspect is recoverable; with twelve, it is not.

A plan for the four hours

TIME	WHAT YOU ARE DOING
0:00–0:30	Triage, cut, and write the plan down
0:30–2:00	Musts — finish the main flow and authentication
2:00–3:00	Shoulds, and every state
3:00–3:30	Deploy, then the full verification battery
3:30–4:00	Documents, README, and rehearse aloud

The last hour is not optional. Verification and documentation are twenty per cent of the mark and the only part guaranteed to be assessed — protect that hour even if the Shoulds are unfinished. Keep the plan visible: it is what stops an interesting refactor eating ninety minutes.

When you are stuck

SITUATION	DO THIS
Stuck for twenty minutes	Ask — the instructor is in the room
A feature will not work	Cut it, and note it in FUTURE.md
The deployment breaks	Roll back, then diagnose
A refactor is tempting	Not today. Write it in FUTURE.md

Twenty minutes, then ask. Struggling alone is how an afternoon disappears, and today there is a person in the room whose job is to unblock you.

Part 3 · The Verification Battery

Every check you have learned across two modules, in one pass on the **deployed** URL. Thirty minutes, and it is the difference between believing the application works and knowing it does.

security — the three attacks, on the live URL

```
// 1 - signed out, from the console
await someWriteAction("<any id>");
// 2 - signed in as someone else
await someWriteAction("<their record id>");
// 3 - a crafted return link
/signin?next=https://example.com
```

All three must be refused, and you must be able to point at the line that refused each — tomorrow, out loud. Also change an id in a private URL and confirm you get a refusal rather than someone else's data, and search the browser bundle for anything you would mind published.

Performance and accessibility

CHECK	HOW	TARGET
LCP, CLS, INP	Lighthouse, throttled, deployed	Under 2.5s, 0.1, 200ms
Keyboard only	Mouse away, Tab and Enter	Every function reachable
Real phone	Open the live URL on it	Usable, nothing hover-only
Images	Network tab	Nothing over about 150 kB
Headings and alt text	View source	One h1, alt on every image

The keyboard pass is the fastest win. Five minutes without a mouse finds more real defects than any automated audit, and the usual finding is a clickable div that should have been a button — fixing it improves the accessibility mark and the code-quality mark at once.

Discovery and correctness

CHECK	LOOKING FOR
View source, deployed	Absolute og:image, one h1
Share the URL to yourself	A card with a picture
/sitemap.xml, /robots.txt	Real routes, no private ones
A brand-new account	Sensible empty states everywhere
A private window	Works with no session or cache

Creating a new account and using the application through it is the single most revealing test — every unwritten empty state appears at once, and an assessor will do exactly this tomorrow. Then clone your own repository into a fresh folder and follow your README: if it does not run, the README is wrong, and that is marked.

The last commit

BEFORE YOU STOP	CHECK
Everything pushed and deployed	git status clean, deployment current
No secret in the repository	Search the history, not just the files
README live URL, four documents	It opens, and they say something true

> **Warning A secret in the history stays there**
> Deleting the file does not remove it from git. If a key was ever committed, rotate it — removing it from the working tree is not enough. And stop building at the end of the session: changes made tonight are untested changes, and a working application you have verified beats a slightly larger one you have not.

Part 4 · Preparing to Present

MINUTES	WHAT TO COVER
0–1	What it is and who it is for — no code
1–3	The main flow, live, from the deployed URL
3–5	One technical decision, and why you made it
5–6	The three attacks, refused in front of the room
6–7	PERF.md — before, after, and the cause
7–8	What you cut, and what you would do next

Start with the person. One sentence on who uses it and what for — an assessor who understands the purpose can follow every technical decision after it, and one who does not is guessing throughout. Rehearse it **aloud, with a timer, today**: it is the only reliable way to discover that the demonstration takes fourteen minutes.

The code walkthrough

HAVE READY TO OPEN	BE READY TO EXPLAIN
One server component	Why it is not a client component
One client component	Why it has to be
A protected action	Both checks inside it
A data query	Where it runs, and why

The walkthrough marks whether you can justify your choices. "It works" is a demonstration; "I put it here because" is a walkthrough. Have the five files open in tabs before you start — searching your own repository under time pressure looks worse than it is.

The questions you will be asked

QUESTION	WHAT IT IS REALLY ASKING
Why is this a client component?	Do you understand the boundary?
What if I call this action signed out?	Is the check in the interface or the code?
Why this rendering strategy?	Did you decide, or accept a default?
What would another week give you?	Do you know what is weak about it?

> **Tip "I do not know" is an acceptable answer**
> Said once, followed by how you would find out, it costs almost nothing. Inventing a justification costs a great deal more, because the follow-up question always arrives — and an assessor can tell the difference immediately.

Tomorrow

Bring the live URL, the repository open, your five files in tabs, the four documents, and eight minutes you have said aloud once. **Expect** to present, to answer questions on your own decisions, to review a peer's work, and to sit the module assessment. **Then** Module 4 begins the backend track — Node with Nest, Go, or .NET — and the frontend you deployed becomes the thing it serves.

Assigned Reading

Required

1. **nextjs.org — Production checklist** What / why: the framework's own list — read it once more against your capstone today.
2. **Make a README** What / why: structure, and the minimum a reader needs to run your project.

Optional — go deeper

- **MoSCoW prioritisation** — Must, Should, Could: the method behind today's triage.
- **GitHub — Removing sensitive data** — what to do if a key was ever committed, and why deleting the file is not enough.

Today's Sequence

Work these in order. Do not skip the last two.

1. Triage into Must, Should and Could; cut the Coulds into FUTURE.md; write the plan down.
2. Fix anything broken, then finish the main flow end to end on the deployed URL.
3. Verify authentication with the three attacks and an id changed in a private URL.
4. Create a brand-new account, use the application through it, and fix every empty state.
5. Run Lighthouse, the keyboard pass and a real phone; record the numbers.
6. Write README.md, AUTH.md, DATA.md, PERF.md, FUTURE.md.
7. Rehearse the eight minutes aloud with a timer, then stop changing the application.

Deliverable · The Capstone

Tip What you submit

One repository with a live URL in the README: the main flow complete, authentication verified by three failed attacks, every state written, the full battery passed on the deployed build, and five short documents.

Check yourself

- Can someone else complete the main task without you explaining anything?
- Did all three attacks fail, and can you name the line that stopped each?
- What does a brand-new account see, and does a fresh clone run from your README?